

Especificación de la API REST

Proyecto Polaflix - Documentación Técnica e Implementación

Carlos Alarcón

26 de abril de 2026

1. Introducción

Este documento detalla la interfaz REST para el sistema **Polaflix**. La API permite una gestión integral del catálogo de series y del ciclo de vida del usuario.

Decisiones de Diseño sobre las Respuestas de la API

En el diseño de esta API, se ha optado por una aproximación pragmática y eficiente en cuanto a los cuerpos de respuesta (payloads), basada en las buenas prácticas de REST y en el patrón de Vistas de Jackson (@JsonView):

- **Cuerpos Vacíos para Acciones de Estado:** Las operaciones que modifican el estado del sistema pero no generan un nuevo recurso consultable inmediatamente (como los métodos DELETE para borrar usuarios o quitar de pendientes, y ciertos PUT como registrar visualizaciones o añadir a pendientes) devuelven un código de estado 200 OK sin cuerpo. Esto reduce la sobrecarga de la red y confirma el éxito de la operación. En caso de error, se informa mediante códigos HTTP adecuados (404 Not Found, 400 Bad Request, etc.).
- **Vistas Dinámicas (DTOs al vuelo):** En las peticiones GET o de creación de recursos, se utilizan las interfaces de Views de Jackson para limitar la profundidad del grafo de objetos serializado. Por ejemplo, al pedir el catálogo de series, se devuelve una vista resumida sin cargar todas las colecciones de temporadas y actores asociadas, optimizando radicalmente el rendimiento.

A continuación se presenta un resumen de los recursos disponibles:

Resumen de Endpoints

- **Usuarios:**
 - GET /usuarios/{username} - Obtener perfil detallado.
 - PUT /usuarios/{username} - Crear o actualizar usuario.
 - DELETE /usuarios/{username} - Eliminar usuario.
 - PUT /usuarios/{username}/series-pendientes/{idSerie} - Añadir a pendientes.
 - DELETE /usuarios/{username}/series-pendientes/{idSerie} - Quitar de pendientes.
 - PUT /usuarios/{username}/capitulos-vistos/{idCapitulo} - Registrar visualización.
- **Series:**

- GET `/series` - Listar catálogo (con filtros de inicial o título).
- GET `/series/{id}` - Obtener detalle completo de una serie.

2. Endpoints de Usuarios

2.1. Obtener Perfil de Usuario

Endpoint: GET /usuarios/{username}

Muestra toda la información del usuario, incluyendo su historial de facturación detallado, capítulos vistos y el estado de seguimiento de cada serie.

Implementación en Java

```
@GetMapping("/{username}")
@JsonView(Views.UsuarioBasico.class)
@Operation(summary = "Obtener perfil de usuario", description = "Devuelve los
    datos del usuario, su plan y el estado de sus series.")
public ResponseEntity<Usuario> obtenerUsuario(@PathVariable("username") String
    username) {
    Usuario usuario = usuarioService.obtenerUsuario(username);

    if (usuario != null) {
        return ResponseEntity.ok(usuario);
    } else {
        return ResponseEntity.notFound().build();
    }
}
```

Ejemplo de Respuesta (Usuario: mrajoy)

```
{
  "username": "mrajoy",
  "facturas": [
    {
      "id": "F-MR-001",
      "mes": 4,
      "anio": 2026,
      "lineas": [
        { "fecha": "2026-04-26", "serieNombre": "Peaky Blinders", "
temporadaCapitulo": "T1xC1", "carga": 1.5 },
        { "fecha": "2026-04-26", "serieNombre": "Peaky Blinders", "
temporadaCapitulo": "T1xC2", "carga": 1.5 }
      ]
    }
  ],
  "iban": { "numeroCuenta": "ES0011112222333344445555" },
  "planSuscripcion": { "tarifaPlana": false, "cuotaMensual": 0 },
  "capitulosVistos": [
    { "id": 1, "numero": 1, "titulo": "Episodio 1", "descripcion": "Thomas
Shelby encuentra por accidente..." },
    { "id": 2, "numero": 2, "titulo": "Episodio 2", "descripcion": "Tommy
desafia a los corredores..." }
  ],
  "estadoSeries": {
    "Prison Break": "EMPEZADA",
    "Peaky Blinders": "EMPEZADA",
    "La que se avecina": "PENDIENTE"
  }
}
```

2.2. Crear o Actualizar Usuario

Endpoint: PUT /usuarios/{username}

Crea un nuevo usuario en el sistema o actualiza los datos de uno existente.

Implementación en Java

```
@PutMapping("/{username}")
@JsonView(Views.UsuarioBasico.class)
@Operation(summary = "Crear o actualizar usuario", description = "Si el usuario no existe, lo crea (Registro). Si existe, actualiza sus datos.")
public ResponseEntity<Usuario> guardarOActualizarUsuario(
    @PathVariable("username") String username,
    @RequestParam String contrasena,
    @RequestParam String iban,
    @RequestParam boolean esTarifaPlana,
    @RequestParam double cuota) {
    try {
        Usuario usuarioActualizado = userService.guardarOActualizarUsuario(
            username, contrasena, iban, esTarifaPlana, cuota);
        return ResponseEntity.ok(usuarioActualizado);
    } catch (Exception e) {
        return ResponseEntity.badRequest().build();
    }
}
```

Ejemplo de Respuesta (Usuario: nuevo_usuario)

```
{
  "username": "nuevo_usuario",
  "facturas": [],
  "iban": { "numeroCuenta": "123123123123" },
  "planSuscripcion": { "tarifaPlana": false, "cuotaMensual": 15 },
  "capitulosVistos": [],
  "estadoSeries": {}
}
```

2.3. Borrar Usuario

Endpoint: DELETE /usuarios/{username}

Elimina la cuenta y todos los datos asociados. No devuelve cuerpo de respuesta.

Implementación en Java

```
@DeleteMapping("/{username}")
@Operation(summary = "Borrar usuario", description = "Elimina permanentemente una cuenta de usuario del sistema.")
public ResponseEntity<Void> borrarUsuario(@PathVariable("username") String username) {
    try {
        userService.borrarUsuario(username);
        return ResponseEntity.ok().build();
    } catch (IllegalArgumentException e) {
        return ResponseEntity.notFound().build();
    }
}
```

2.4. Añadir a Pendientes

Endpoint: PUT /usuarios/{username}/series-pendientes/{idSerie}

Añade una serie a la lista del usuario. Sin cuerpo de respuesta.

Implementación en Java

```
@PutMapping("/{username}/series-pendientes/{idSerie}")
@Operation(summary = "Añadir a pendientes", description = "Añadir una serie al
cat logo personal del usuario en estado PENDIENTE.")
public ResponseEntity<Void> agregarSeriePendiente(
    @PathVariable("username") String username,
    @PathVariable("idSerie") String idSerie) {
    try {
        usuarioService.agregarSeriePendiente(username, idSerie);
        return ResponseEntity.ok().build();
    } catch (IllegalArgumentException e) {
        return ResponseEntity.notFound().build();
    }
}
```

2.5. Quitar de Pendientes

Endpoint: DELETE /usuarios/{username}/series-pendientes/{idSerie}

Elimina una serie de la lista de pendientes. Sin cuerpo de respuesta.

Implementación en Java

```
@DeleteMapping("/{username}/series-pendientes/{idSerie}")
@Operation(summary = "Quitar de pendientes", description = "Elimina una serie
de la lista del usuario (solo si est en estado PENDIENTE).")
public ResponseEntity<Void> quitarSeriePendiente(
    @PathVariable("username") String username,
    @PathVariable("idSerie") String idSerie) {
    try {
        usuarioService.quitarSeriePendiente(username, idSerie);
        return ResponseEntity.ok().build();
    } catch (IllegalArgumentException e) {
        return ResponseEntity.notFound().build();
    }
}
```

2.6. Registrar Visualización

Endpoint: PUT /usuarios/{username}/capitulos-vistos/{idCapitulo}

Marca un capítulo como visto y realiza los cargos de facturación necesarios. Sin cuerpo de respuesta.

Implementación en Java

```
@PutMapping("/{username}/capitulos-vistos/{idCapitulo}")
@Operation(summary = "Marcar capítulo como visto", description = "Registra que
un usuario ha visto un capítulo y genera cargos en su factura si
corresponde.")
public ResponseEntity<Void> verCapitulo(
    @PathVariable("username") String username,
    @PathVariable("idCapitulo") Long idCapitulo) {
    try {
```

```
        usuarioService.verCapitulo(username, idCapitulo);
        return ResponseEntity.ok().build();
    } catch (IllegalArgumentException e) {
        return ResponseEntity.notFound().build();
    } catch (Exception e) {
        return ResponseEntity.internalServerError().build();
    }
}
```

3. Endpoints de Series

3.1. Obtener Catálogo de Series

Endpoint: GET /series

Permite obtener la lista de series. Soporta filtrado por inicial o por título.

Implementación en Java

```
@GetMapping
@JsonView(Views.SerieResumida.class)
@Operation(summary = "Obtener cat logo", description = "Devuelve el cat logo de series. Permite filtrar por inicial o por fragmento de t tulo (solo uno a la vez).")
public ResponseEntity<List<Serie>> obtenerCatalogo(
    @RequestParam(required = false) String inicial,
    @RequestParam(required = false) String titulo) {

    if (inicial != null && titulo != null) {
        return ResponseEntity.badRequest().build();
    }

    List<Serie> catalogo = serieService.obtenerSeries(inicial, titulo);

    if (!catalogo.isEmpty()) {
        return ResponseEntity.ok(catalogo);
    } else {
        return ResponseEntity.noContent().build();
    }
}
```

Ejemplo: Filtrado por inicial "P"

```
[
  {
    "id": "S01",
    "titulo": "Peaky Blinders",
    "inicial": "P",
    "sinopsis": "Una familia de gansteres asienta su dominio...",
    "costeVisionado": 1.5,
    "categoria": "GOLD"
  },
  {
    "id": "S02",
    "titulo": "Prison Break",
    "inicial": "P",
    "sinopsis": "Michael Scofield elabora un plan magistral...",
    "costeVisionado": 0.75,
    "categoria": "SILVER"
  }
]
```

Ejemplo: Filtrado por título

```
[
  {
    "id": "S01",
    "titulo": "Peaky Blinders",
    "inicial": "P",
    "sinopsis": "Una familia de gansteres asienta su dominio...",
  }
]
```

```

    "costeVisionado": 1.5,
    "categoria": "GOLD"
  }
]

```

3.2. Obtener Detalle de Serie

Endpoint: GET /series/{id}

Devuelve toda la información de la serie, incluyendo creadores, actores y todas sus temporadas con sus respectivos capítulos.

Implementación en Java

```

@GetMapping("/{id}")
@JsonView(Views.SerieDetallada.class)
@Operation(summary = "Obtener detalle de serie", description = "Devuelve los detalles completos de una serie, incluyendo temporadas, cap tulos y actores.")
public ResponseEntity<Serie> obtenerDetalleSerie(@PathVariable("id") String id)
{
    Serie serie = serieService.obtenerSeriePorId(id);

    if (serie != null) {
        return ResponseEntity.ok(serie);
    } else {
        return ResponseEntity.notFound().build();
    }
}

```

Ejemplo de Respuesta (Serie: S03 - La que se avecina)

```

{
  "id": "S03",
  "titulo": "La que se avecina",
  "inicial": "L",
  "sinopsis": "Las disparatadas vidas y problemas cotidianos...",
  "creadores": [
    { "id": 18, "nombre": "Alberto Caballero" }
  ],
  "actores": [
    { "id": 13, "nombre": "Jordi Sanchez" },
    { "id": 17, "nombre": "Pablo Chiapella" }
  ],
  "temporadas": [
    {
      "id": 9,
      "titulo": "Temporada 1 de La que se avecina",
      "numero": 1,
      "descripcion": "Descripcion pendiente...",
      "capitulos": [
        { "id": 49, "numero": 1, "titulo": "Mirador de Montepinar", "descripcion": "Los nuevos vecinos llegan..." },
        { "id": 50, "numero": 2, "titulo": "Okupas, secuestros y un golpe", "descripcion": "El caos se apodera..." }
      ]
    }
  ]
  /* Se omiten temporadas restantes en este ejemplo por brevedad */
},
"costeVisionado": 0.5,
"categoria": "ESTANDAR"

```

}
